COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	8 / 18 / 26	**DATE SUBMITTED:**	8 / 26 / 26
NAME:	Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 6.1: Searching Techniques

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main (){
    // Generate random values
    int dataset[max_size];
    srand(time(0));

    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    int userInput;
```

Observations

The cstdlib library is used for random number generation and time.h as a seeding function. Using the const int max_size, the dataset array is limited to contain 50 random numbers. Using a for loop, random numbers are put inside the array and then printed afterwards.

Table 6-1. Data Generated and Observations.

Code

main:

```
    int userInput;

    // Testing Array using LS
    std::cout << "What number to search?\n";
    std::cin >> userInput;
    std::cout << "Array result LS: ";

    linearSearchArr(max_size, userInput, dataset);
```

header:

```
template <class T>
void linearSearchArr(int arrSize, T dataFind, T list[]){

    // Step 1: I = 0

    // Repeat while I <= n
    for(int i = 0; i < arrSize; i++){
        if(dataFind == list[i]){
            // Print "Searching is successful"
            std::cout << "Searching is successful\n";
            return;
        }
    }

    // Print "Searching is unsuccessful"
    std::cout << "Searching is unsuccessful\n";
}
```

Output

```
C:\Users\user\OneDrive\Docu
28723 578 32656 22501 24089 24085 9669 14713 14600
 22924 27865 7445 13294 1768 23767 28198 16854 115
50 174 11376 16836 21549 4406 31489 8672 27566 208
46 7011 15831 16301 8 10518 31871 12215 551 10557
28858 23604 11111 17048 412 15864 18206 12722 2401
6 23572 18145 19804 15927 5106
What number to search?
412
Array result LS: Searching is successful
```

Observations	It looks for a number that is the same by using a for loop by using the maximum size of the array to search the array from the start until it triggers the if condition as the number becomes the same as the one that the user inputs.

Table 6-2a. Linear Search for Arrays

Code

main:

```
// Create Linked List: S A N T O S
Node<char>* name1 = new_node('S');
Node<char>* name2 = new_node('A');
Node<char>* name3 = new_node('N');
Node<char>* name4 = new_node('T');
Node<char>* name5 = new_node('O');
Node<char>* name6 = new_node('S');

// Linking
name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;

// Testing LL using LS
std::cout << "Linked List result LS: \n";
std::cout << "For 'R': ";
linearSearchLL(name1, 'R');

std::cout << "\n For 'O': ";
linearSearchLL(name1, 'O');
std::cout << std::endl;
```

header:

```
template <class T>
void linearSearchLL(Node<T>* head, T dataFind){
    while(head != NULL){
        if(head->data == dataFind){
            std::cout << "Searching is successful\n";
            return;
        }
        head = head->next;
    }
    std:: cout << "Searching is unsuccessful\n";
}
```

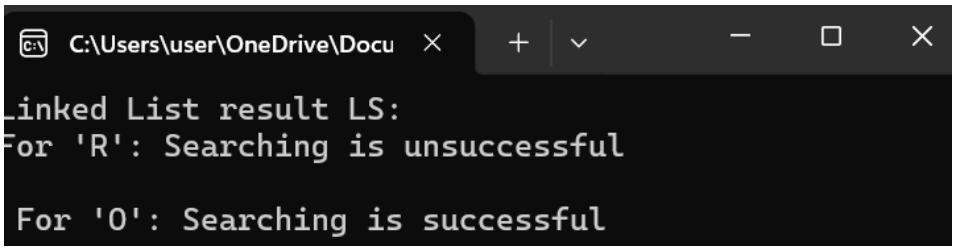
Output	
Observations	It searches for a char by starting from the first node and moving to the next node one by one until it triggers the if condition when the char becomes the same as the one being searched for.

Table 6-2b. Linear Search for Linked List

Code	**main:**
	`// Testing Array using BS`
	`int listArrBS[10] = {5, 12, 18, 23, 31, 42, 57, 64, 78, 91};`
	
	`std::cout << "\nBinary Search Array:\n";`
	`binarySearchArr(10, 42, listArrBS);`
	`binarySearchArr(10, 100, listArrBS);`
	
	header:
	`template <class T>`
	`void binarySearchArr(int arrSize, T dataFind, T list[]){`
	`  // Define low, up and middle`
	`  int low = 0, up = arrSize-1, middle;`
	
	`  // Loop while low is <= up`
	`  while(low <= up){`
	`  // Compute middle`
	`  middle = (int)((low+up)/2);`
	
	`  // If data found is middle -> data found -> return`
	`  if(dataFind == list[middle]){`
	`  std::cout << "Search element is found!\n";`
	`  return;`
	`  }`
	
	`  // If data find < data middle -> new high`
	`  else if(dataFind < list[middle]){`
	`  up = middle-1;`
	`  }`
	`  else{`
	`  // Else update low = middle + 1`
	`  low = middle + 1;`
	`  }`
	`  }`
	`  std::cout << "Search element is not found! \n";`
	`}`

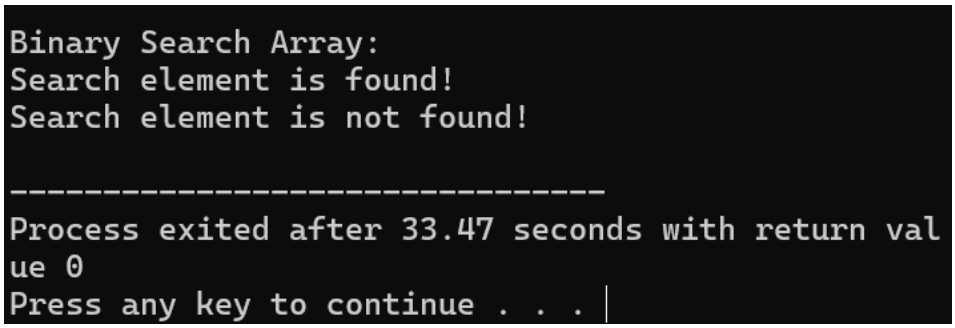
Output	
Observations	It looks for a number by using the middle of an array and comparing it with the number being searched for and then repeatedly reducing the search range to either the left or right side of the array.

Table 6-3a. Binary Search for Arrays

Code	**main:**
	`// Testing LL using BS`
	`std::cout << "Linked List result BS: \n";`
	`std::cout << "\n For 'S': ";`
	`binarySearchLL(name4, 'S');`
	
	`std::cout << "\n For 'E': ";`
	`binarySearchLL(name3, 'E');`
	
	header:

```
template <class T>
void binarySearchLL(Node<T>* start, T dataFind){

    // Step 1: Set start head of the list, set last to null
    Node<T>* last = NULL;

    // Continue search while search range is valid (Not yet last)
    while(start != last){

    // Step 2: Finding middle using getMiddle
    Node<T>* middle = getMiddle(start, last);

    // Step 3: Data is found
    if(middle->data == dataFind){
        std::cout << "Searching is successful\n";
        return;
    }

    // Step 4: Searching left
    else if(middle->data > dataFind){
        last = middle;
    }

    // Step 5: Searching right
    else{
        start = middle->next;
    }
}
    // Step 6: Print display
    std::cout << "Searching is unsuccessful\n";
}
```

Output

```
Linked List result BS:

 For 'S': Searching is successful

 For 'E': Searching is unsuccessful
```

Observations

It looks for a char by using a node in the middle of the linked list to see which side of the list to search and then starts to search the selected side.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'? 2 Comparisons.

Code

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 10;

int main (){

    // Problem 1
    int dataset[max_size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

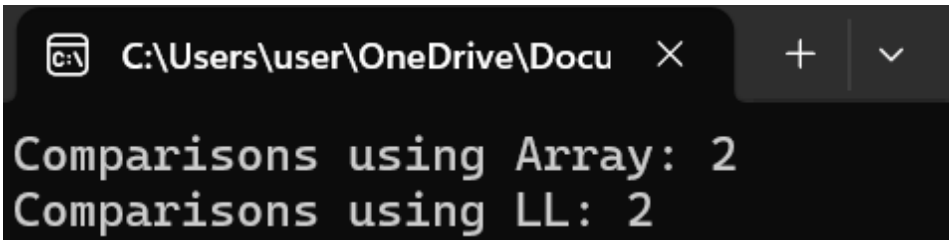
    // Search for 18
    int comp = 0;
    for(int i = 0; i < max_size; i++){
        comp++;

        if(dataset[i] == 18){
            break;
        }
    }
    std::cout << "Comparisons using Array: " << comp << "\n";

    // Create Linked List
    Node<int>* int1 = new_node(15);
    Node<int>* int2 = new_node(18);
    Node<int>* int3 = new_node(2);
    Node<int>* int4 = new_node(19);
    Node<int>* int5 = new_node(18);
    Node<int>* int6 = new_node(0);
    Node<int>* int7 = new_node(8);
    Node<int>* int8 = new_node(14);
    Node<int>* int9 = new_node(19);
    Node<int>* int10 = new_node(14);

    int1->next = int2;
    int2->next = int3;
    int3->next = int4;
    int4->next = int5;
    int5->next = int6;
    int6->next = int7;
    int7->next = int8;
    int8->next = int9;
    int9->next = int10;

    // LL search
    std::cout << "Comparisons using LL: ";
    std::cout << lSLLCount(int1, 18) << "\n\n";
```

Output	

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code

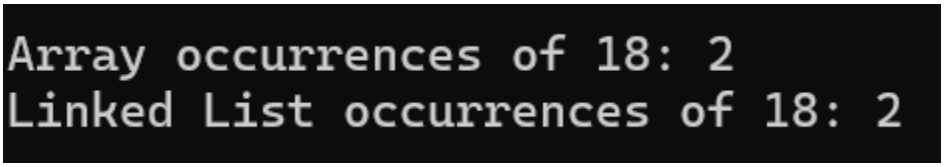
```
// Problem 2
// Count repetetion in Array
int count = 0;

for(int i = 0; i < max_size; i++){
    if(dataset[i] == 18){
        count++;
    }
}
std::cout << "Array occurrences of 18: " << count << std::endl;

// Counting repetition in LL
count = 0;
Node<int>* current = int1;

while(current != NULL){
    if(current->data == 18){
        count++;
    }
    current = current->next;
}
std::cout << "Linked List occurrences of 18: " << count << "\n\n";
```

Output



Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you want to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Code

```
// Problem 3
int sorted[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
std::cout << "Sorted Array: ";
for(int i = 0; i < max_size; i++){
    std::cout << sorted[i] << " ";
}
std::cout << std::endl;

// Binary Search for 8
int low = 0; int up = max_size - 1; int middle; int rep_counter = 1;
while(low <= up){

    // Compute middle
    middle = (int)((low + up) / 2);

    std::cout << "Repetition " << rep_counter << " - ";
    std::cout << " Low: " << low;
    std::cout << " Up: " << up;
    std::cout << " Middle: " << middle;
    std::cout << " Value: " << sorted[middle] << "\n";

    // If data found is middle
    if(8 == sorted[middle]){
        std::cout << "Search element is found\n";
        break;
    }

    // If data find is less than data middle, then set as new high
    else if(8 < sorted[middle]){
        std::cout << "Left side search:\n";
        up = middle - 1;
    }

    else{
        // Else update low to middle + 1
        std::cout << "Right side search:\n";
        low = middle + 1;
    }
    rep_counter++;
}
```

Output

```
Sorted Array: 3 5 6 8 11 12 14 15 17 18
Repetition 1 -  Low: 0 Up: 9 Middle: 4 Value: 11
Left side search:
Repetition 2 -  Low: 0 Up: 3 Middle: 1 Value: 5
Right side search:
Repetition 3 -  Low: 2 Up: 3 Middle: 2 Value: 6
Right side search:
Repetition 4 -  Low: 3 Up: 3 Middle: 3 Value: 8
Search element is found
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code

```
    // Problem 4
    // Recursive Binary Search
    if(bSRecursive(0, max_size - 1, 8, sorted)){
        std::cout << "Search element is found\n";
    }
    else{
        std::cout << "Search element is not found\n";
    }

    return 0;
}
```

Output

```
Search element is found

--------------------------------
Process exited after 1.313 seconds with return value 0
Press any key to continue . . .
```

Conclusion:

I learned how both types of searching work and the difference between linear searching and binary searching and applying them to an array and a linked list. I find linear search easier to do since it can be straightforward and can be done with one for loop and I see that binary search is hard since it requires more conditions and address manipulation but I was able to understand the two searching techniques but I still need to improve my programming of binary search since it is harder and more difficult when applying recursion on the binary search.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.